

Benchmarking On-Policy vs. Off-Policy Reinforcement Learning for Complex Visual Tasks: A Case Study in Minecraft Wood Collection

Ally Descoteaux, Stone Liu, Cameron Ryan, Alison Ryan, Lucas Kroon

Abstract

This project explores the efficacy of Deep Reinforcement Learning (RL) agents in navigating the high-dimensional, open-world environment of Minecraft to perform the fundamental task of wood collection. We compare the performance of two industry-standard algorithms: Proximal Policy Optimization (PPO), an on-policy method, and Soft Actor-Critic (SAC), an off-policy method. Utilizing raw pixel inputs and raycasting for visual attention, we developed a custom environment to train agents in diverse biomes. Our results indicate that PPO significantly outperforms SAC in the early stages of training (within the first 1,000,000 timesteps), demonstrating greater stability and a higher rate of positive experience sampling. While SAC shows signs of delayed learning as the replay buffer populates with successful interactions, PPO's on-policy approach proves more efficient for the complex, sparse-reward landscape of Minecraft survival tasks.

Introduction

1.1 Problem Statement

In the field of Artificial Intelligence, Minecraft serves as a premier "sandbox" for Reinforcement Learning due to its procedurally generated worlds, complex physics, and sparse rewards. The primary challenge of this project is to train an autonomous agent from scratch to identify, navigate to, and collect wood, a fundamental survival task, using only raw visual data and raycast-based attention. This requires the agent to map high-dimensional pixel input to discrete actions while overcoming the "needle in a haystack" problem of finding reward-generating interactions in a vast environment.

1.2 Background and Motivation

Traditional RL methods, such as tabular Q-Learning and Deep Q-Networks (DQN), frequently struggle with the continuous state spaces and long-horizon goals found in 3D environments. Initial phases of this research confirmed that these methods were infeasible for Minecraft due to the sheer scale of the state-action space. Consequently, we shifted our focus to modern Actor-Critic architectures.

The core of our investigation lies in the trade-off between on-policy and off-policy learning:

- **PPO (Proximal Policy Optimization):** Known for stability and reliability by ensuring policy updates do not deviate too far from the previous iteration.
- **SAC (Soft Actor-Critic):** Known for sample efficiency by reusing past experiences stored in a replay buffer, though often more sensitive to hyperparameter tuning.

1.2.1 On-Policy Approaches (PPO)^{3,4,5,6}

Across the literature, the application of unmodified PPO in sparse rewards environments displays a consistent pattern: zero-reward trajectories caused by sparse rewards and the sample inefficiency inherent in PPO's structure are prone to causing premature policy collapse. When this happens the agent is unable

to easily discover the high-reward states that would yield a useful gradient signal. This makes properly tuning the algorithm’s hyperparameters a rather strenuous and delicate task in these environments; as such, PPO is often augmented using some form of external structure.

Broadly speaking, three major strategies have been used for this purpose. The first is intrinsic motivation, in which curiosity-based signals reward the visitation of new states and compensate for the lack of intrinsic feedback. In this case, the agent’s behavior can be quite sensitive to the strength of the intrinsic signal and has a tendency to distract the agent from the true objective. The second is demonstration-guided learning, in which the agent receives external guidance in the form of biased policy updates derived from small sets of sparse-reward demonstrations; this helps the agent to move more quickly through the initial stage of exploration using only a few trajectories. The third is auxiliary-reward scaffolding, in which separate policies are trained on shaped rewards and guide the main agent towards optimizing for the true objective; this borrows the sample efficiency of shaped learning while still preserving the alignment of the final policy with the intended goal.

1.2.2 Off-Policy Approaches (SAC)^{7,8,9,10,11}

The off-policy nature of SAC as well as its focus on maximizing randomness and exploration give it an obvious structural advantage over PPO; the rare reward signals available in the environment are able to be frequently revisited in future episodes through the use of replay buffers. In the literature, however, there is a consensus that these clear advantages are not sufficient for more difficult sparse-rewards tasks leading to a similar strategy of augmentation. The dominant approaches parallel those of PPO but make an effort to utilize the off-policy nature of SAC to its advantage.

The first, auxiliary-task scheduling, concurrently trains multiple intention policies and exploits shared off-policy experiences across tasks; this method has proven especially effective in robotic manipulation problems which have environments similar to our own focus. The second is learned reward shaping which uses preference learning and semi-supervised techniques to train secondary rewards functions from the sparse signal of the environment to use in place of the original reward function. The third is demonstration-based extensions which integrate imperfect demonstrations into SAC’s value estimation or use discriminator-augmented Q-functions to allow agents to surpass demonstrator performance rather than being bounded by it.

1.2.3 Precedent for Further Work

While the literature shows a clear interest in the application of both PPO and SAC in sparse reward environments, there is a very clear gap in comparative studies. Often, authors will make reasoned arguments that guide them toward exploring the application of one or the other and forgo the valuable insight to be gained by testing both frameworks in the same task such that direct comparisons can be made. Any existing benchmarks focus on low-dimensional continuous control or highly specialized domains such as Atari games and driving simulators; this leaves a clear gap around visually complex and partially observed environments like Minecraft.

1.3 Summary of Methods

Our framework utilizes a custom API to interface with Minecraft, allowing for headless training on a High-Performance Computing (HPC) cluster. We integrated a Convolutional Neural Network (CNN) to process pixel-based screenshots, supplemented by raycasting to simulate visual focus on specific blocks. We employed a "shaped reward" strategy, incentivizing the agent not just for the final collection of wood, but for looking at trees and actions that related to wood collection. To benchmark performance, we conducted "apples-to-apples" comparisons by training PPO and SAC concurrently in identical environments (Forest and Taiga biomes) for 500,000 to 1,000,000 timesteps.

1.4 Summary of Results

Experimental data reveals a stark contrast in learning curves. PPO demonstrated immediate and consistent improvement, achieving significantly higher rewards within the first 60,000 steps compared to SAC. Specifically, PPO completely outperforms SAC in early-stage efficiency, likely because off-policy sampling from a replay buffer is inefficient when positive experiences are initially rare. Our findings suggest that for complex, visually-driven tasks with high-dimensional inputs, on-policy algorithms provide a more robust starting point, though off-policy methods may hold potential once a baseline of success is established.

Methodology

Environment

The input to each model is a 12 x 64 x 64 tensor representing four 3 x 64 x 64 pixel inputs. The output of SAC is a single vector of 8 continuous numbers between -1 and 1 representing the 8 distinct actions we allow the agent to perform. These include: camera pitch/yaw, navigation in all 4 directions such as forward, backward, left, and right, attack, and jump. Each is thresholded such that positive values map to a discrete button press which is acceptable for MineRL. The main objective of the agent is to collect as much wood as possible without any prior pretraining from the MineRL dataset. Furthermore, we opted to not use the standard **MineRLTreechop-v0** in favor of a custom reward environment using ray-tracing and sparse rewards. Each episode terminated after at most 2,000 environment steps. A successful episode threshold was set to 100 reward or roughly 50 logs chopped. Every episode was reset to a different forest seed and the agent is given a starting inventory of a diamond axe to help it in its primary objective. Most notably our integration of the ray tracing helped the agent pick up reward signals from the environment when it was looking at favorable blocks such as wood or leaves. The reward signals are as follows:

Signal	Value	Note
Time Penalty	-0.0002	Tiny penalty for the agent when standing still.
Log Mined	2.0	Applies for any wood type.
Leaf Broken	0.0001	Applies for any leaf type.
Dirt Mined	-0.002	Discourages agents from

		digging into the ground.
Grass Mined	-0.002	Discourages agents from digging grass.

Each agent was trained from 500,000 to 1,000,000 timesteps through both standard forest and taiga biomes on a HPC (High Performance GPU Cluster).

Soft Actor Critic

Soft Actor Critic is an off-policy, maximum entropy reinforcement learning algorithm. It attempts to maximize the following objective function:

$$J(\pi) = \sum_t \mathbb{E} [r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot|s_t))]$$

Throughout training, our team ran multiple SAC models with different parameters. The ones we decided to focus on were the buffer size and gradient steps. Larger buffer sizes enabled a more stable sampling of state/action experiences throughout training while larger gradient steps enabled the model to extract more reward signal despite the sparse reward environment. The following hyperparameters for each run of SAC can be found here:

Hyperparameter	SAC Main V2	SAC Fine Tuned V6	SAC Taiga V1
Buffer Size	500000	1000	500000
Gamma (Discount Factor)	0.99	0.99	0.99
Gradient Steps	8	1	8
Learning Rate	3.00E-04	3.00E-04	3.00E-04
Batch Size	512	256	512

Proximal Policy Optimization

Proximal Policy Optimization (PPO) is an on-policy actor-critic reinforcement learning algorithm designed to ensure stable and efficient policy updates. Unlike off-policy methods, PPO updates its policy using only data collected from the current policy, which improves training stability in high-dimensional and sparse-reward environments such as Minecraft. It essentially tries to maximize the following clipped surrogate objective that approximates expected return:

$$L^{CLIP}(\theta) = \mathbb{E}_t [\min (r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t)]$$

In our implementation, PPO operates on the same observation space as SAC: a stacked tensor of pixel inputs ($12 \times 64 \times 64$) combined with raycasting-derived visual attention signals. The policy network

consists of a convolutional neural network (CNN) that extracts spatial features from raw pixel data, followed by fully connected layers that output action probabilities over the discrete action space.

PPO optimizes a clipped surrogate objective function, which constrains policy updates to remain within a bounded region of the previous policy. This prevents large, destabilizing updates that could degrade performance. Additionally, we use Generalized Advantage Estimation (GAE) to reduce variance in policy gradient updates while maintaining bias control.

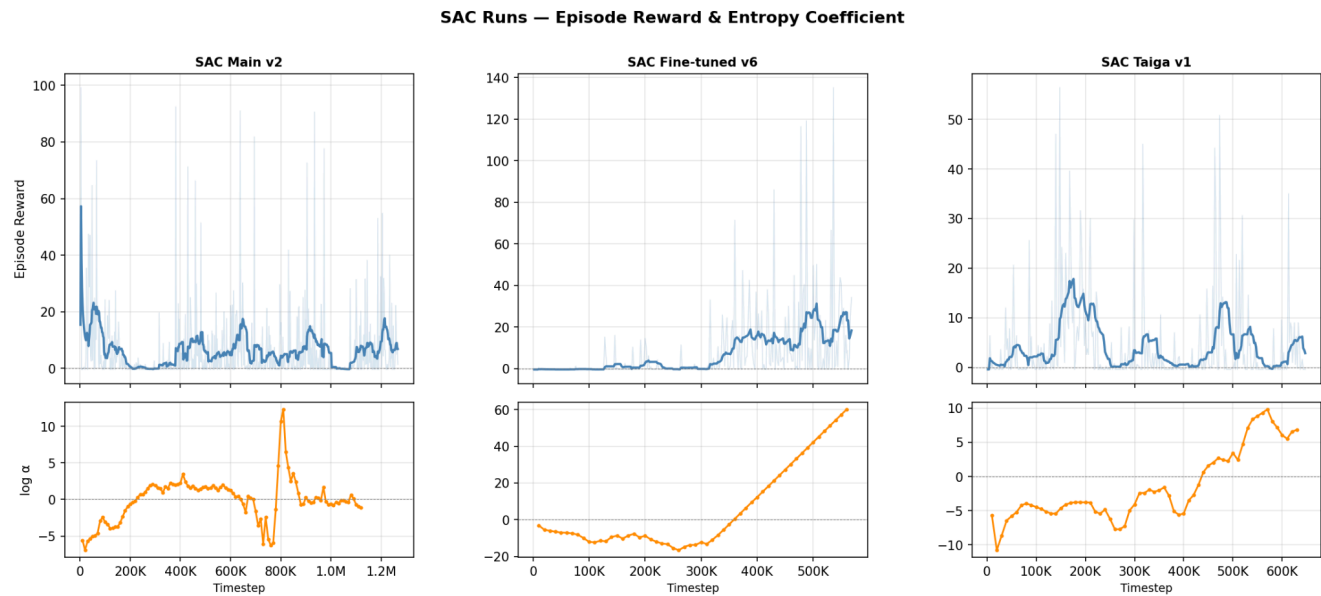
Throughout training, our team also ran multiple PPO models with different parameters, with the most important changes being to the learning rate and clip range. Higher learning rates allowed our model to take bigger steps each update and learn faster, but could cause instability/overshooting. A larger clip range allows for larger policy updates but is less stable and could cause the policy to collapse. We found the most success using a learning rate of 1E-4 and clip range of .2.

Hyperparameter	PPO V1	PPO V2	PPO V3
Learning Rate	6E-5	1E-4	1E-4
Clip Range	0.1	0.3	0.2

Models, Analysis, and Results

Soft Actor Critic

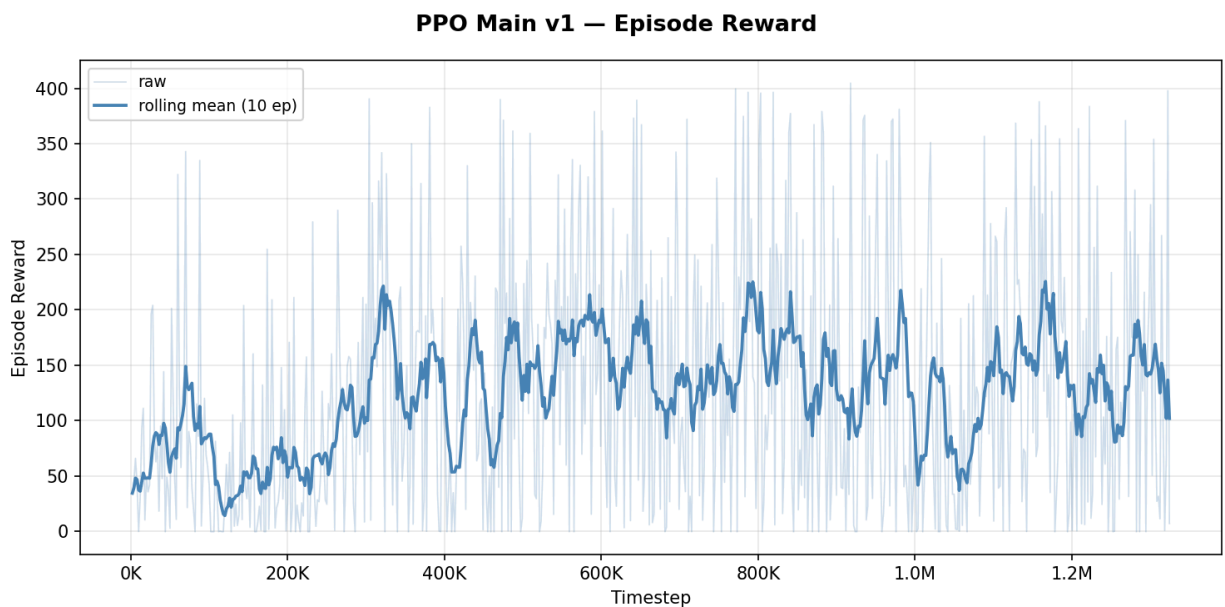
Our team trained Soft Actor Critic with various hyperparameters across 500,000 to 1,000,000 timesteps. The results from SAC shows that it failed to extract and learn from the reward signal provided from the environment.

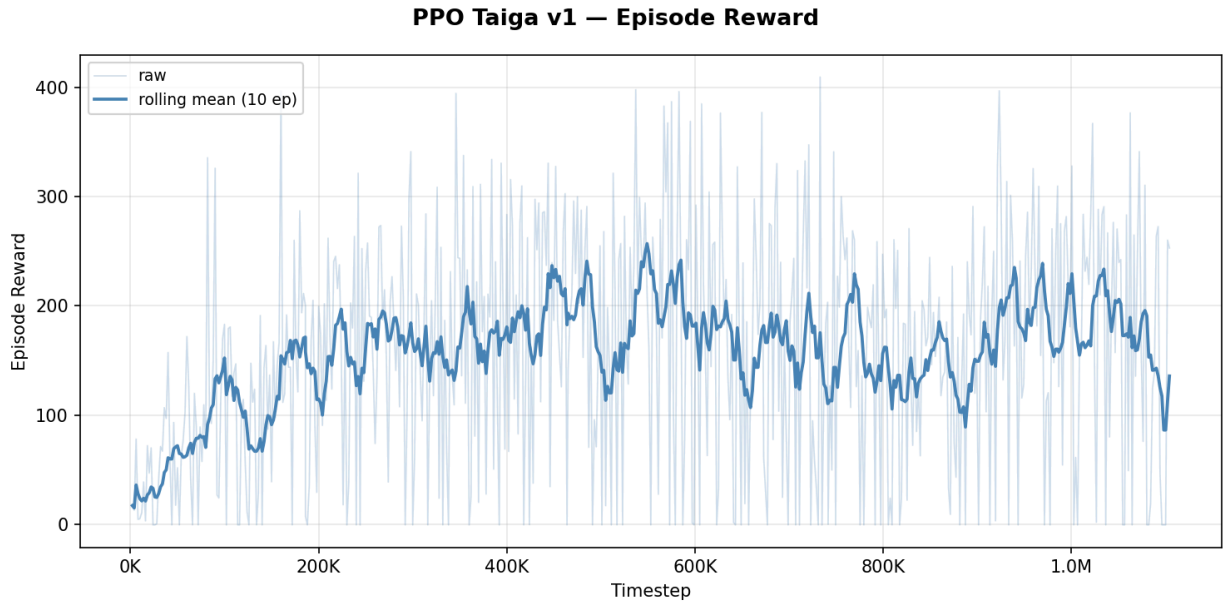


SAC agents appear to start off with very little entropy resulting in a highly deterministic policy from the beginning. As the agent gathers more experiences, the entropy coefficient grows larger suggesting that the agent fails to converge to an optimal policy and favors more exploration to escape local minima. Average wood collected did not exceed past 3 per episode across all training runs. SAC agents across runs appear to not have extrapolated the reward signal well enough to converge as evidenced by the high variability of rewards that often collapsed to negative or zero reward throughout episodes.

Proximal Policy Optimization

The performance of PPO across both experimental environments demonstrates consistent learning, stability, and strong convergence behavior.





In both environments, PPO exhibits a clear upward learning trend during the early stages of training. Within approximately the first 100k–200k timesteps, the rolling average reward increases rapidly from near-zero values to over 100 reward per episode, indicating that the agent successfully begins to identify and exploit reward-generating behaviors such as locating and harvesting wood.

Reinforcement Learning Framework

The project evaluates two primary Deep Reinforcement Learning (RL) architectures:

- **Proximal Policy Optimization (PPO):** An on-policy algorithm that maintains training stability by clipping policy updates to prevent drastic changes in agent behavior.
- **Soft Actor-Critic (SAC):** An off-policy, maximum entropy algorithm designed for sample efficiency through the use of a replay buffer.

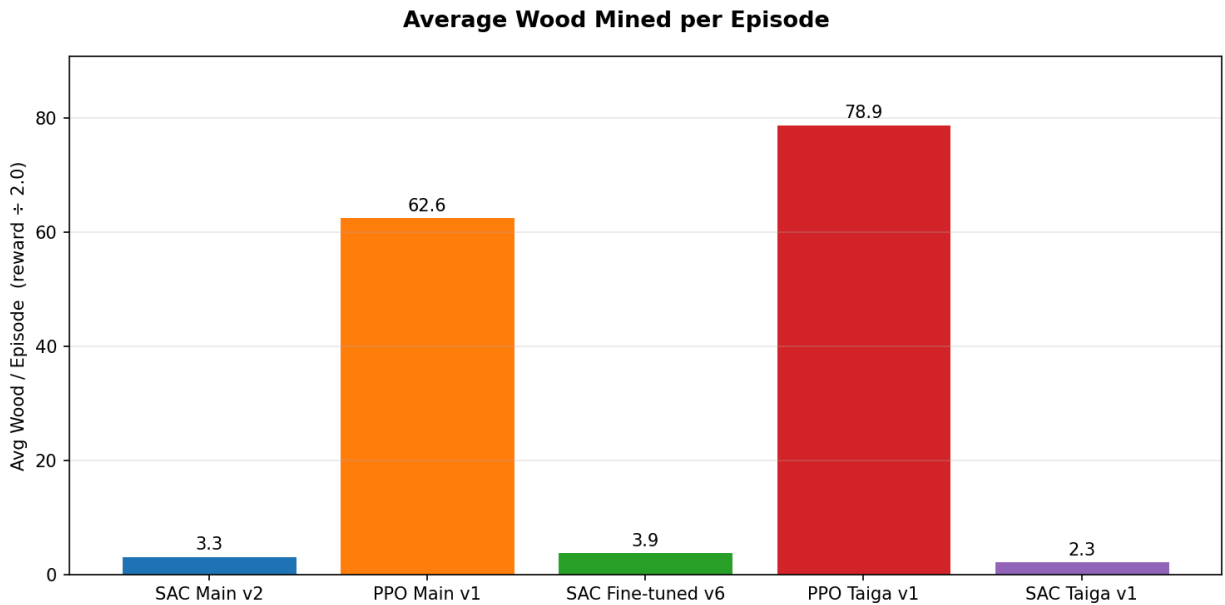
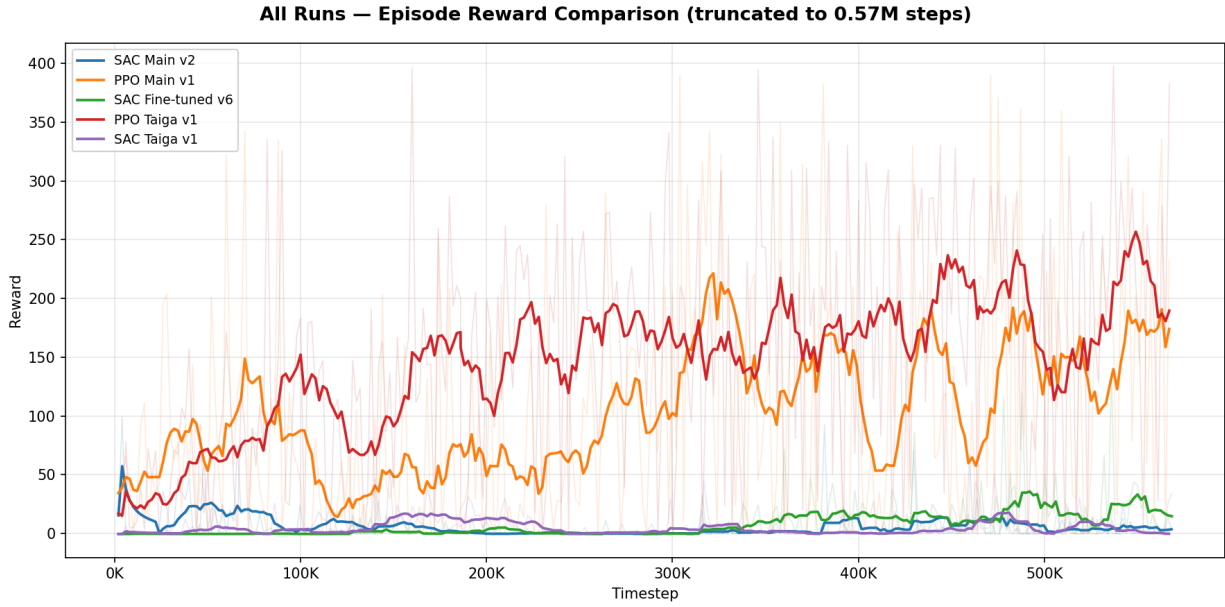
Model Input: Agents were trained using raw pixel data from the Minecraft environment, augmented by a raycasting system to provide the agent with focused visual attention on blocks within its line of sight.

Experimental Setup and Hyperparameters

Training was conducted on a High-Performance Computing (HPC) cluster. We compared five distinct runs across different biomes (Forest and Taiga) and configurations:

Parameter	SAC Main v2	PPO Main v1	SAC Fine-tuned v6	PPO Taiga v1	SAC Taiga v1
Algorithm	SAC	PPO	SAC	PPO	SAC
Learning Rate	3.00E-04	1.00E-04	3.00E-04	1.00E-04	3.00E-04

Batch Size	512	256	256	256	512
Total Timesteps	1266367	1324302	568513	1104536	646896
Avg Wood/Episode	3.27	62.62	3.9	78.88	2.29
Gamma	0.99	0.995	0.99	0.995	0.99
Buffer Size	500000	—	1000	—	500000
Learning Starts	500	—	0	—	500
Tau	0.005	—	0.005	—	0.005
Gradient Steps	8	—	1	—	8
Ent Coef	auto	0.01	auto	0.01	auto
Target Entropy	-8	—	-8	—	-8
Target Update Int	1	—	1	—	1
Train Freq (steps)	4	—	1	—	4
Replay Buf n_steps	10	—	1	—	10
Optimize Memory	FALSE	—	FALSE	—	FALSE
Use SDE	FALSE	FALSE	FALSE	FALSE	FALSE
N Steps	—	2048	—	2048	—
N Epochs	—	10	—	10	—
GAE Lambda	—	0.95	—	0.95	—
Clip Range	—	0.2	—	0.2	—
VF Coef	—	0.5	—	0.5	—
Max Grad Norm	—	0.5	—	0.5	—
Normalize Advantage	—	TRUE	—	TRUE	—
Clip Range VF	—	None	—	None	—
Target KL	—	None	—	None	—



Results and Comparative Analysis

The empirical data shows a significant performance gap between the two algorithms:

- PPO Performance:** In both the Main and Taiga biome runs, PPO exhibited rapid convergence and high stability. As seen in the *Episode Reward Comparison*, PPO agents quickly reached a reward plateau between **150 and 250**, with some episodes peaking near **400**. PPO Taiga v1 (red line) showed the most aggressive learning curve, effectively identifying and targeting wood blocks early in the training process.

- **SAC Performance:** SAC struggled significantly in this sparse-reward environment. The *SAC Main v2* and *SAC Taiga v1* runs remained near a reward of 0 for the vast majority of the 1.2M timesteps.
- **Ablation (SAC Fine-tuning):** The *SAC Fine-tuned v6* run (green line) showed a slight improvement over the baseline SAC models after 400k steps, suggesting that hyperparameter adjustments—specifically reducing the buffer size—may help SAC eventually learn, though it still failed to reach PPO's performance within the tested timeframe.

Discussion

On-Policy vs. Off-Policy Efficiency

The clearest takeaway from our experiments is that PPO dramatically outperformed SAC across every configuration we tested. On the surface this seems to contradict SAC's reputation for sample efficiency, but the result makes sense once you consider what "sample efficiency" actually requires: a replay buffer that contains enough useful experiences to learn from.

In Minecraft, an untrained agent has no reliable way to stumble onto a reward. It needs to visually locate a tree, close the distance, and swing at it, a sequence that a randomly initialized policy will almost never execute correctly. PPO benefits directly from this reality: every time the agent does something right, that trajectory immediately shapes the next policy update. The feedback loop is tight by design. SAC, on the other hand, stores every experience in a replay buffer and samples from it at random. In the early stages of training, that buffer is almost entirely made up of wandering, empty episodes. The rare successes get diluted into a sea of zeros, and the resulting gradient updates carry very little meaningful signal.

This played out clearly in the entropy dynamics we observed during SAC training. Rather than gradually converging toward a stable strategy, the entropy coefficient kept climbing, meaning the agent was becoming *more* random over time, not less. It was casting a wider net in search of rewards it couldn't consistently find, which is the opposite of learning. By the end of training, average wood collected across all SAC runs never broke 3.9 logs per episode. PPO, by comparison, reached 62.62 in the forest environment and 78.88 in the taiga.

The fine-tuned SAC run (v6) is worth examining here. By shrinking the buffer size from 500,000 down to 1,000, we forced the agent to sample almost exclusively from recent experience. This produced a small but real improvement after around 400,000 steps, which tells us something: the composition of what SAC trains on matters at least as much as how much data it has access to. That said, even with this adjustment SAC never came close to PPO's performance, and the reward curves remained unstable throughout, suggesting the agent had not settled into anything resembling a coherent strategy.

Environmental Generalization

Performance held up reasonably well across both biomes. PPO Taiga v1 showed a slightly steeper early learning curve than the forest run, but both converged to similar reward ranges. This is a good sign for the

raycasting reward system; differences in wood texture, canopy density, and lighting between biomes did not seem to meaningfully confuse the agent's ability to identify and target trees. It suggests the attention mechanism would likely transfer to other environments without significant modification.

Technical Limitations

The way we adapted SAC to work with Minecraft's discrete action space, thresholding continuous outputs to simulate button presses, may have put it at a structural disadvantage. SAC's exploration strategy is built around the idea that small differences in continuous actions matter; when those actions get collapsed into binary on/off states, some of that expressiveness is lost. It is possible that a properly discrete algorithm would have fared differently.

The 2,000-step episode cap is another factor. For PPO this was not a serious constraint, since positive experiences were being collected and used within each update cycle. For SAC, shorter episodes mean fewer opportunities per episode to generate the successful transitions the replay buffer needs. Finally, both algorithms were trained entirely from scratch. The literature on SAC in sparse-reward environments fairly consistently finds that some form of warm-starting, whether through demonstrations, auxiliary tasks, or a shaped reward curriculum, is often necessary to get off-policy methods off the ground. Our results fit squarely within that pattern.

Conclusions

We set out to test a fairly simple question: between PPO and SAC, which algorithm is better suited for learning wood collection in Minecraft from raw pixels? The answer, at least within the training budget we used, was unambiguous. PPO learned quickly and stably across both biomes, reaching average rewards as high as 78.88 logs per episode. SAC never exceeded 3.9 and showed no clear signs of convergence even after 1.2 million timesteps.

That said, we do not think this means SAC is the wrong tool for environments like Minecraft in any absolute sense. The issue is more specific: SAC's strengths depend on a replay buffer that contains enough successful transitions to learn from, and in a sparse-reward environment like this one, building up that buffer takes far longer than the window in which PPO has already found a working policy. SAC was not failing because it is a worse algorithm, it was failing because the conditions it needed to thrive were not present early enough.

The ablation results from SAC v6 suggest there is room to address this. Faster buffer turnover helped, if only marginally. More targeted interventions, prioritized experience replay, demonstration seeding, or auxiliary reward shaping, could plausibly close the gap more substantially, and testing those approaches is the most natural next step for this work.

Beyond that, two questions feel worth pursuing. First, does SAC eventually catch up if training runs long enough? The entropy trends we observed leave open the possibility that the buffer simply needed more

time to accumulate useful experience, and a run beyond 2 million steps would help settle that. Second, would a pre-trained visual encoder change the picture? A significant portion of what makes this task hard is the raw pixel-to-action mapping problem. If both algorithms were given a head start on the visual side, it might reveal a different balance between them on the decision-making side.

For now, the practical recommendation is straightforward: if you are training a Minecraft agent from scratch under a limited compute budget, PPO is the more reliable starting point. Its on-policy structure means every successful interaction counts immediately, which turns out to matter a great deal when successful interactions are rare.

References

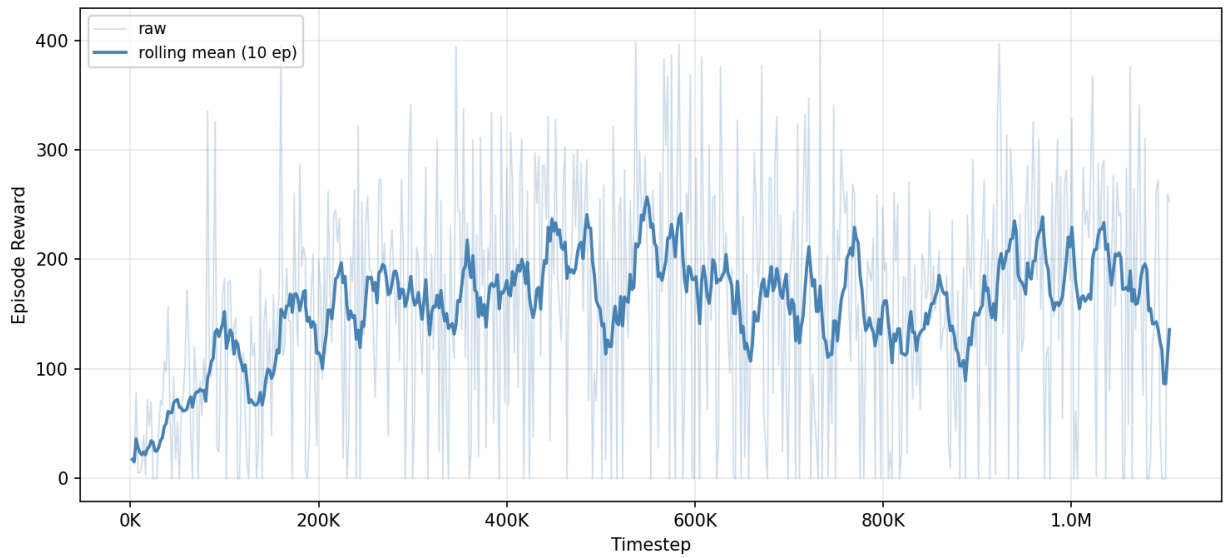
- (1) Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *arXiv preprint arXiv:1801.01290*. <https://arxiv.org/abs/1801.01290>
- (2) Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*. <https://arxiv.org/pdf/1707.06347>
- (3) Chen, Weitong, and Lei Li. (2024) “A Study of PPO Algorithms Combining Curiosity and Imitation Learning in Maze Environments.” 2024 2nd International Conference on Artificial Intelligence and Automation Control (AIAC) [Guangzhou, China], pp. 411–16. DOI.org (Crossref), <https://doi.org/10.1109/AIAC63745.2024.10899618>.
- (4) Hare, Joshua. (2019) “Dealing with Sparse Rewards in Reinforcement Learning.” arXiv:1910.09281, arXiv.org, <https://doi.org/10.48550/arXiv.1910.09281>.
- (5) Huang, Shengyi, and Santiago Ontañón. (2020) “Action Guidance: Getting the Best of Sparse Rewards and Shaped Rewards for Real-Time Strategy Games.” arXiv:2010.03956, arXiv.org, <https://doi.org/10.48550/arXiv.2010.03956>.
- (6) Wang, Guojian, et al. (2024) “Policy Optimization with Smooth Guidance Learned from State-Only Demonstrations.” arXiv:2401.00162, arXiv.org, <https://doi.org/10.48550/arXiv.2401.00162>.
- (7) Lee, Heeseong, et al. (2025) “SAC(λ): Efficient Reinforcement Learning for Sparse-Reward Autonomous Car Racing Using Imperfect Demonstrations.” 2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) [Hangzhou, China], pp. 13460–67. DOI.org (Crossref), <https://doi.org/10.1109/IROS60139.2025.11246186>.
- (8) Li, Wenyun, et al. (2026) “Shaping Sparse Rewards in Reinforcement Learning: A Semi-Supervised Approach.” arXiv:2501.19128, arXiv.org, <https://doi.org/10.48550/arXiv.2501.19128>.
- (9) Memarian, Farzan, et al. (2021) “Self-Supervised Online Reward Shaping in Sparse-Reward Environments.” arXiv:2103.04529, arXiv.org, <https://doi.org/10.48550/arXiv.2103.04529>.
- (10) Riedmiller, Martin, et al. (2018) “Learning by Playing - Solving Sparse Reward Tasks from Scratch.” arXiv:1802.10567, arXiv.org, <https://doi.org/10.48550/arXiv.1802.10567>.
- (11) Vasan, Gautham, et al. (2024) “Revisiting Sparse Rewards for Goal-Reaching Reinforcement Learning.” arXiv:2407.00324, arXiv.org, <https://doi.org/10.48550/arXiv.2407.00324>.

Appendices:

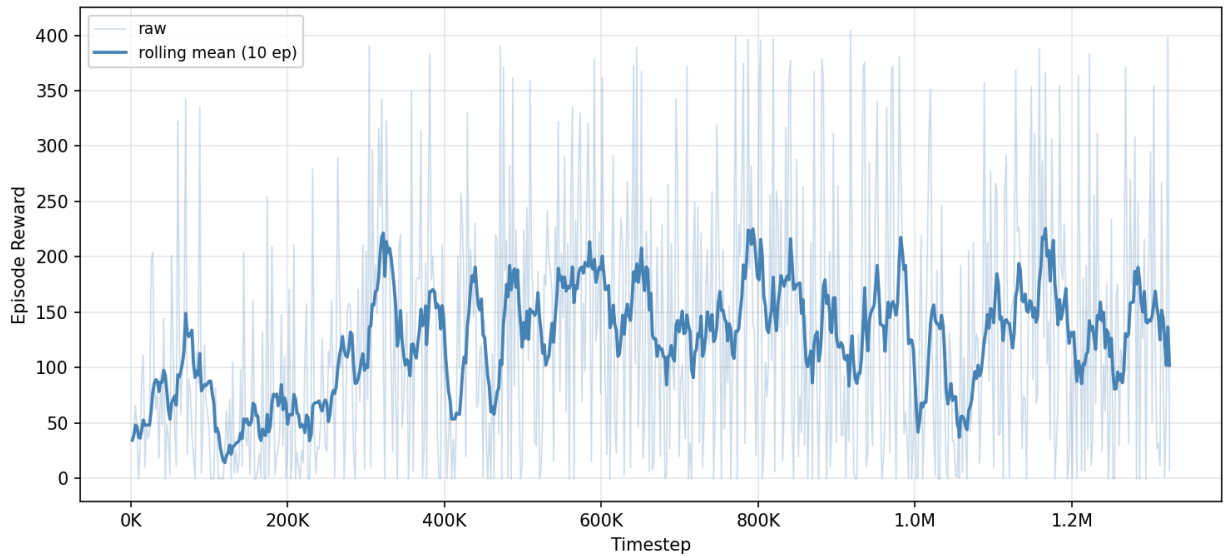
Data sets, codes, brief description of common methods, auxiliary materials

Appendix A (Algorithm Deep Dives)

PPO Taiga v1 — Episode Reward

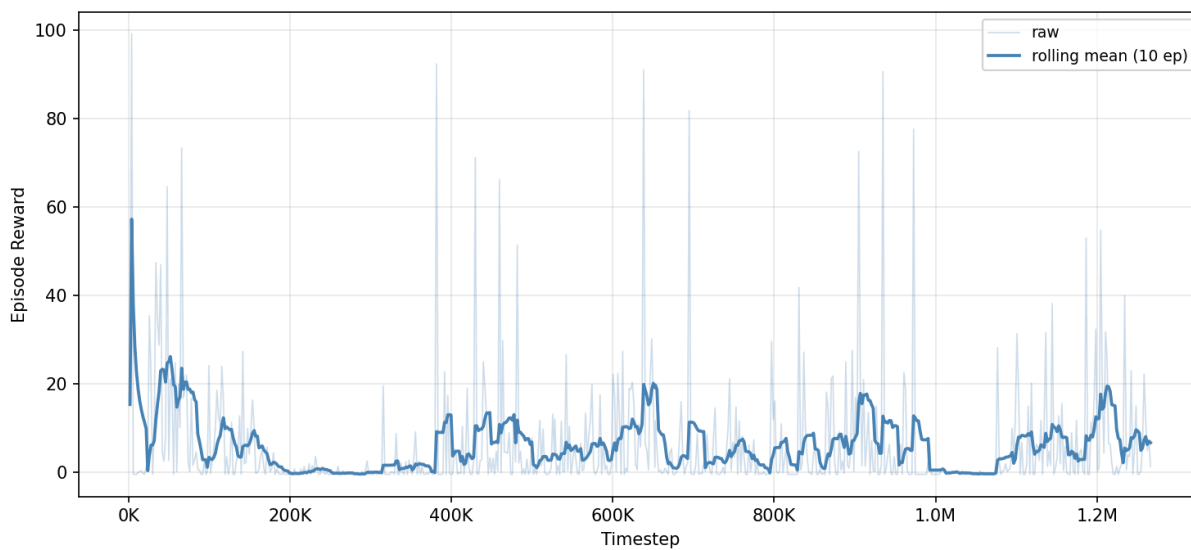


PPO Main v1 — Episode Reward

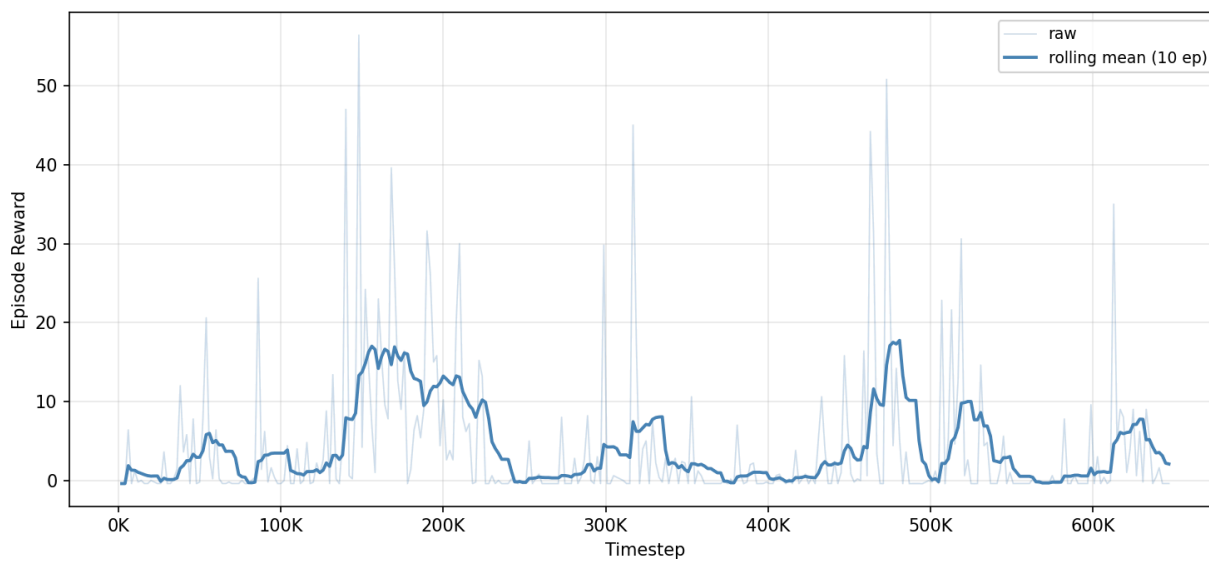


Appendix B (Ablation Studies & Failed Runs):

SAC Main v2 — Episode Reward



SAC Taiga v1 — Episode Reward



SAC Fine-tuned v6 — Episode Reward

